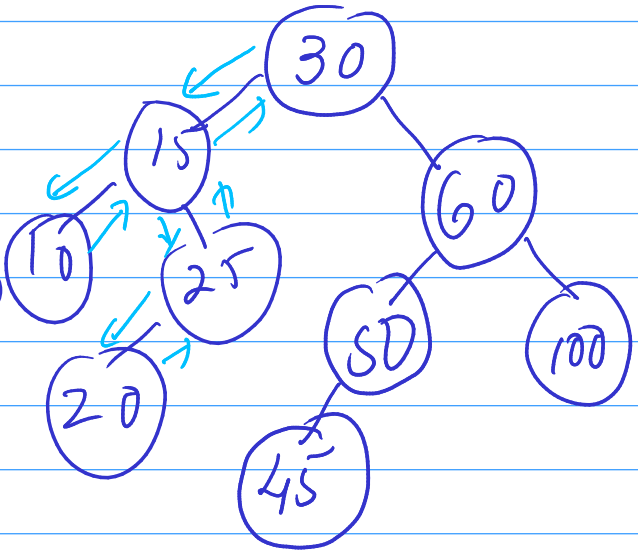
15/04

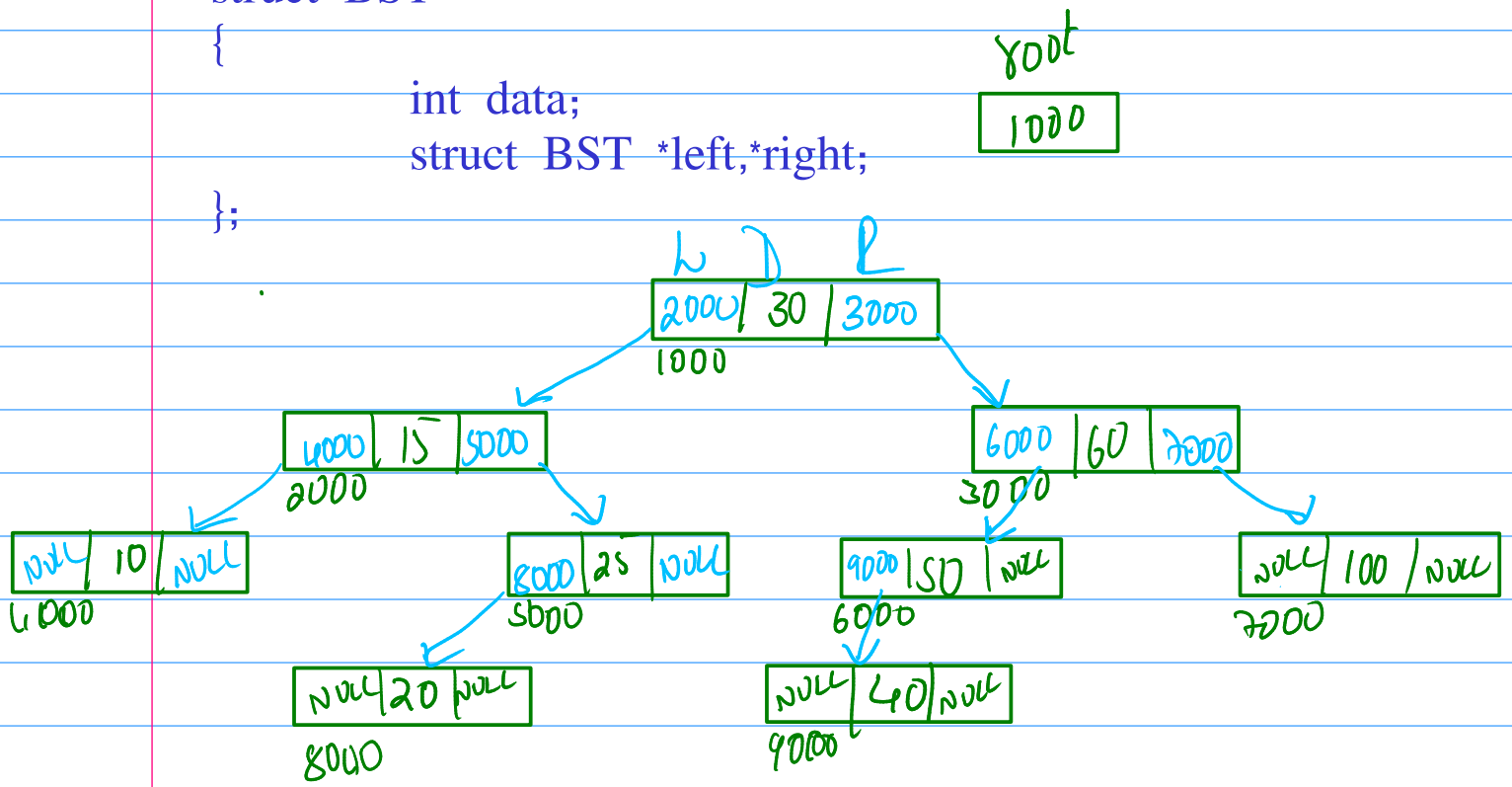
BST insertion

operations :
insert
traverse
search
delete

30,60,50,45,100,15,25,20,10

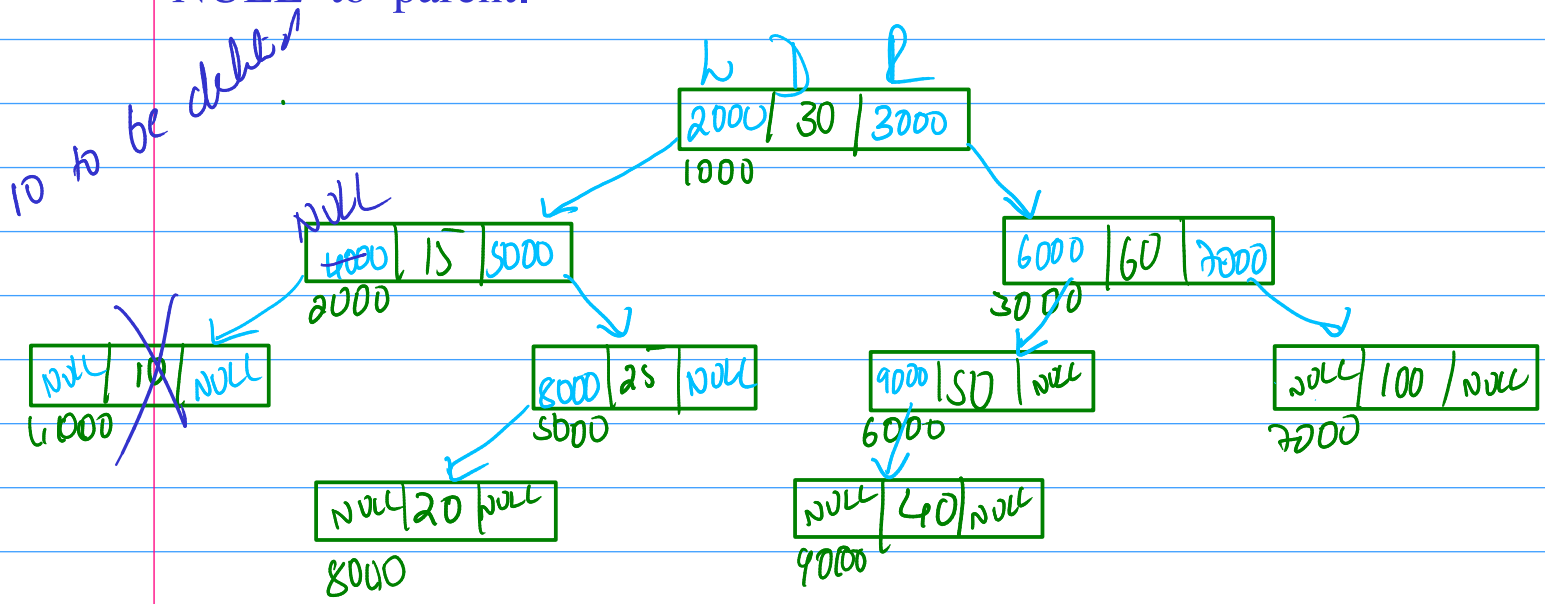



```
struct BST
{
        int data;
        struct BST *left,*right;
};
```

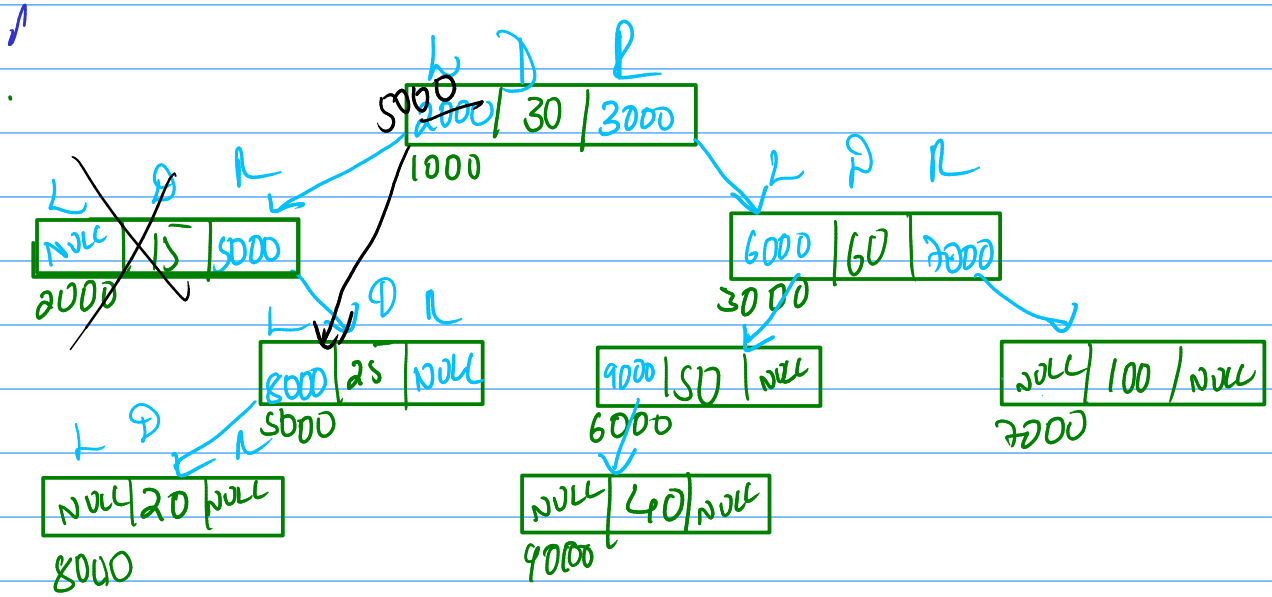


Delete the data from BST

case 1: data matching with leaf node,delete the nodfe and return NULL to parent.



case 2 : data matched with node with Single child.
delete the node and return its single child address to paren

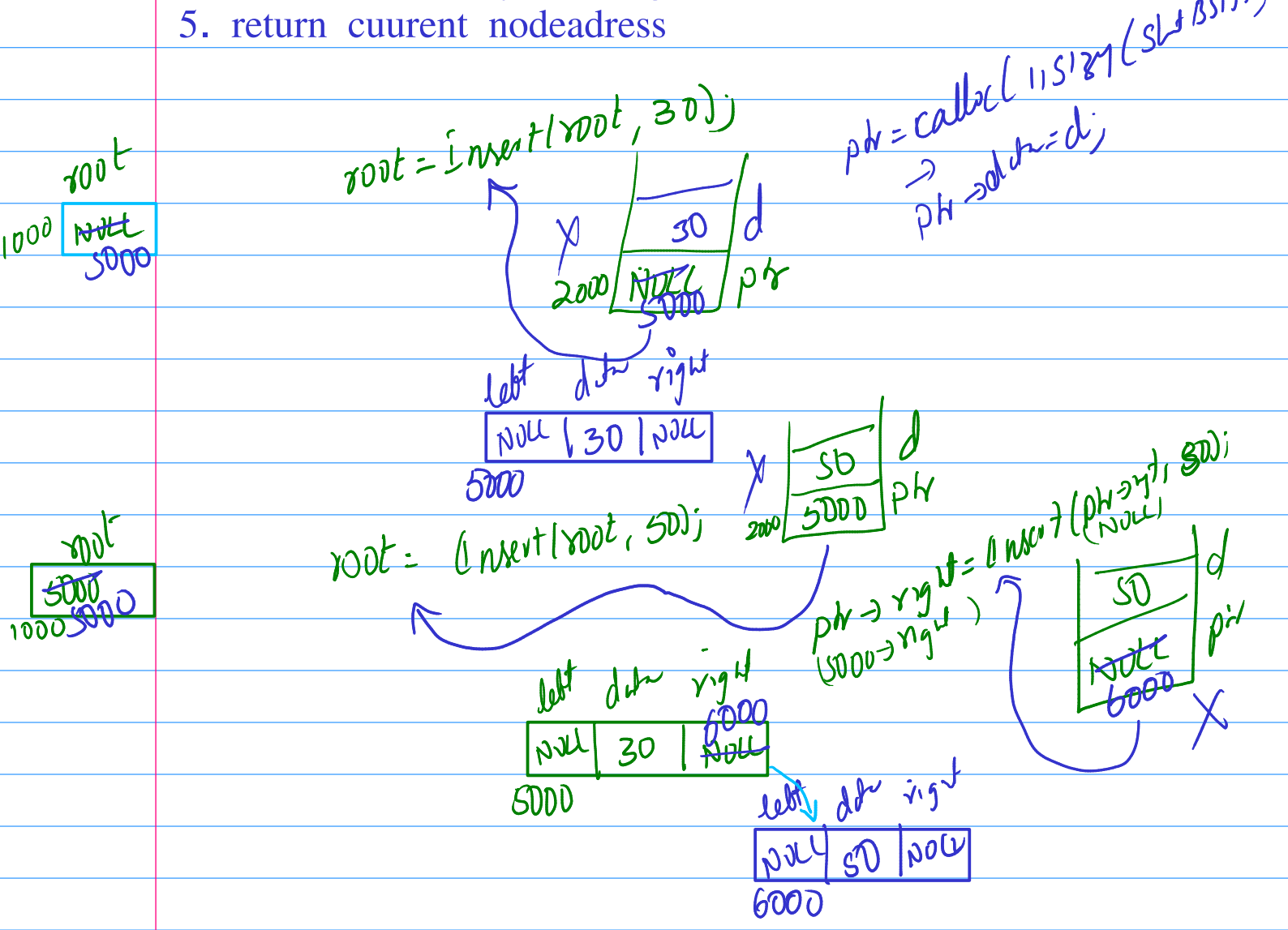


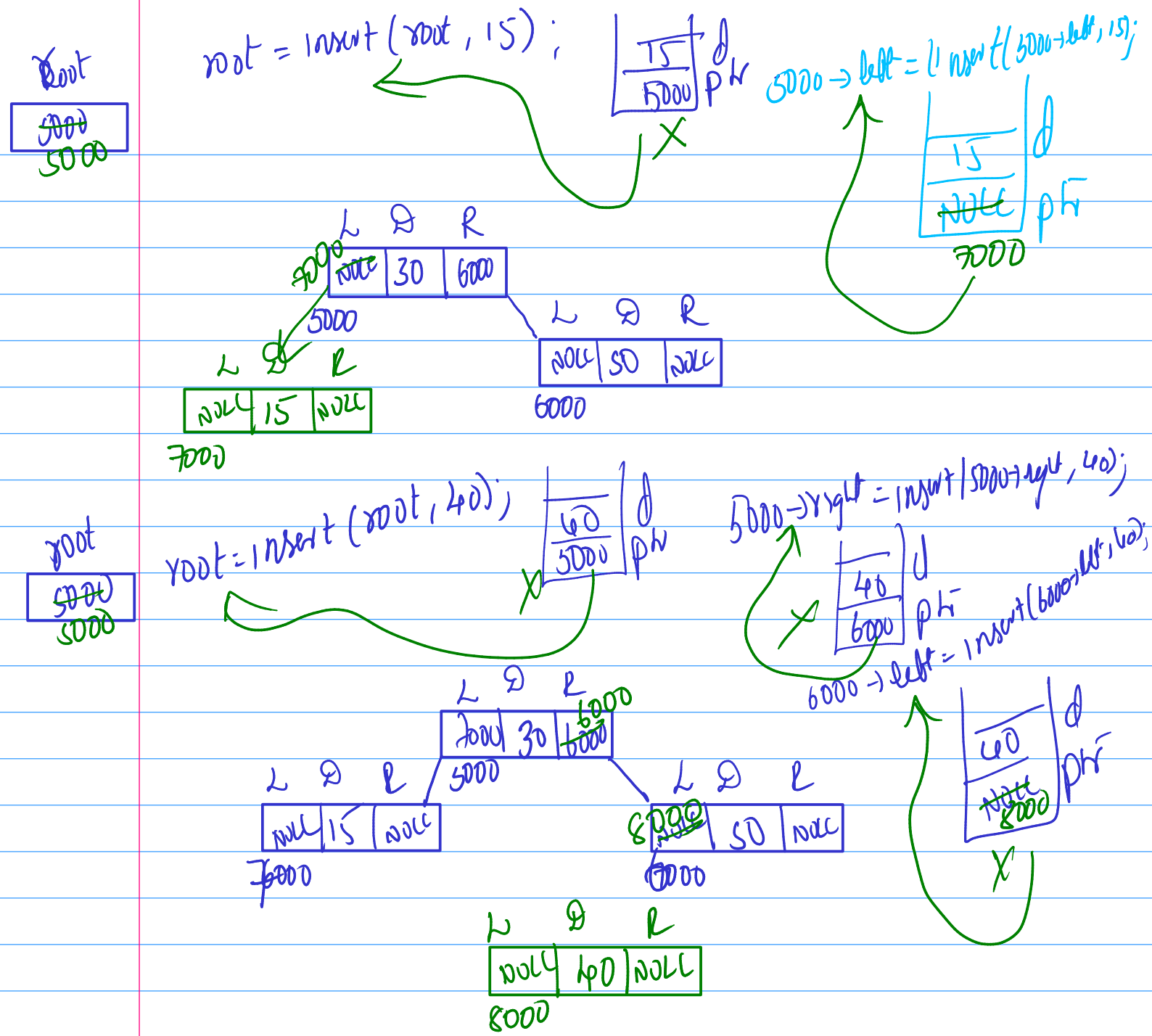
case 3: data matching with node with 2 children.
from Left subtree node with highest value is capable to become the parent , so make it parent and delete theat node from left subtree

From right subtree node with minimum value is capable to become a parent , make it parent and delete that node from right subtree

algorithm for insertion of node in bst tree

1. if current position is vacant , create a node and iniatialise with given data.
2. else if given data is smaller than current node data , travers to left.
3. ~~else if given data is greater then current node data, traverse~~ to right
4. else data already existing
5. return cuurent nodeadress






```
if(ptr == NULL)
{ ptr = calloc(1, sizeof(BST));
  ptr->data = d;

else if (d < ptr->data)
{ ptr->left = insert(ptr->left, d);

else if (d > ptr->data)
{ ptr->right = insert(ptr->right, d);

else duplicate data
```